

```

import { webSearchTool, Agent, AgentInputItem, Runner, withTrace } from
"@openai/agents";

import { z } from "zod";

// Tool definitions

const webSearchPreview = webSearchTool({
  searchContextSize: "medium",
  userLocation: {
    country: "GB",
    type: "approximate"
  }
})

const IntakeScopingSchema = z.object({ objectives: z.string(), constraints: z.string(),
kpis: z.string(), hypotheses: z.string() });

const ResearcherSchema = z.object({ evidence_json: z.string(), citations_str: z.string()
});

const intakeScoping = new Agent({
  name: "Intake & Scoping",
  instructions: `You are the Intake & Scoping consultant.

Parse {{ input_as_text }} into objectives, constraints, KPIs, and hypotheses.

Return STRICT JSON with these string fields containing bullet lists separated by
newlines:

{
  \"objectives_str\": \"...\",
  \"constraints_str\": \"...\",
  \"kpis_str\": \"...\",
  \"hypotheses_str\": \"...\"
} `,

```

```

model: "gpt-4.1",
outputType: IntakeScopingSchema,
modelSettings: {
  temperature: 1,
  topP: 1,
  maxTokens: 2048,
  store: true
}
});

```

```

const researcher = new Agent({
  name: "Researcher",
  instructions: `You are the Researcher.

```

Use the hypotheses below to find 3–5 pieces of external evidence. Each item must have: claim, short support quote (or paraphrase), and URL.

Hypotheses:

```
{{ Intake & Scoping.hypotheses_str }}
```

If tools are available, call `web_search` and `web_read` as needed.

Return STRICT JSON embedded as a single string field (because this workflow uses STR outputs):

```

{
  "evidence_json": "[{"claim": "...", "support": "...", "url": "https://..."},
  ...]"
}

```

Also include a second field listing the URLs (newline separated):

```

{
  "citations_str": "https://...\nhttps://..."
}

```

```
}`,  
  model: "gpt-4.1",  
  tools: [  
    webSearchPreview  
  ],  
  outputType: ResearcherSchema,  
  modelSettings: {  
    temperature: 1,  
    topP: 1,  
    maxTokens: 2048,  
    store: true  
  }  
});
```

```
const analystCsvDataInsightsSimulated = new Agent({  
  name: "Analyst (CSV Data Insights — Simulated)",  
  instructions: `You are the Data Analyst consultant.
```

Use the following simulated datasets to analyse client performance and extract 3–5 key findings that relate to the hypotheses below.

Hypotheses:

```
{{ Intake & Scoping.hypotheses_str }}
```

External evidence:

```
{{ Researcher.evidence_json }}
```

web_analytics.csv (sample)

Date, Sessions, Conversions

2025-07-01, 12000, 240

2025-08-01, 10000, 180

2025-09-01, 9500, 190

2025-10-01, 11000, 230

product_availability.csv (sample)

Date, InStockRate

2025-07-01, 96%

2025-08-01, 84%

2025-09-01, 88%

2025-10-01, 93%

media_spend.csv (sample)

Channel, Spend, ROI

Search, 15000, 1.4

Social, 10000, 1.2

Display, 5000, 0.9

Tasks:

1. Interpret trends across these datasets.
2. Quantify relationships (e.g., stockouts vs conversions, spend vs ROI).
3. Produce a short Markdown report with a table of key findings and a bullet list of suggested charts.

4. Return your result in ****strict JSON**** so downstream nodes can still read it:

```
{
  \"report_md\": \"Full Markdown report including table and bullet list\",
  \"findings_str\": \"newline-separated findings for other nodes\",
  \"charts_str\": \"newline-separated chart ideas\"
}
`,
model: \"gpt-4.1\",
modelSettings: {
  temperature: 1,
  topP: 1,
  maxTokens: 2048,
  store: true
}
});
```

```
const strategyLead = new Agent({
  name: \"Strategy Lead\",
  instructions: `You are the Strategy consultant.
```

Use these findings to define 2–3 strategic options and pick one recommendation.

Findings:

```
{{ Analyst.findings_str }}
```

Return strict JSON:

```
{
```

```
\ "options_str\ ": \ "newline-separated list of options\ ",  
\ "recommendation_str\ ": \ "single-sentence recommendation\ "  
} ` ,  
model: "gpt-4.1",  
modelSettings: {  
  temperature: 1,  
  topP: 1,  
  maxTokens: 2048,  
  store: true  
}  
});
```

```
const pmo = new Agent({  
  name: "PMO",  
  instructions: ` You are the PMO consultant.
```

From the recommendation below, produce a concise 30/60/90 plan and the top 3 risks.
Return STRICT JSON only (no prose, no code fences), with these fields:

```
{  
  \ "plan_30d\ ": \ "One sentence for the first 30 days\ ",  
  \ "plan_60d\ ": \ "One sentence for days 31–60\ ",  
  \ "plan_90d\ ": \ "One sentence for days 61–90\ ",  
  \ "risk_1\ ": \ "Top risk in one sentence\ ",  
  \ "risk_2\ ": \ "Second risk in one sentence\ ",  
  \ "risk_3\ ": \ "Third risk in one sentence\ ",  
  \ "plan_str\ ": \ "Bulleted or newline version of the full 30/60/90 for downstream use\ ",  
  \ "risks_str\ ": \ "Bulleted or newline list of risks for downstream use\ ",
```

```
\ "summary_str\": \ "One-paragraph summary for humans (optional)\ "  
}
```

Recommendation:

```
{{ \ "Strategy Lead\ ".recommendation_str }}` ,  
  model: "gpt-4.1",  
  modelSettings: {  
    temperature: 1,  
    topP: 1,  
    maxTokens: 2048,  
    store: true  
  }  
});
```

```
const qaEthics = new Agent({  
  name: "QA & Ethics",  
  instructions: ` You are the QA & Ethics consultant.  
  Review all previous content for factual soundness, ethical alignment, and internal  
  consistency.
```

Inputs:

Findings:

```
{{ Analyst.findings_str }}
```

Recommendation:

```
{{ Strategy Lead.recommendation_str }}
```

Plan:

```
{{ PMO.plan_str }}
```

Risks:

```
{{ PMO.risks_str }}
```

Tasks:

1. Check for unsupported or conflicting claims.
2. Flag any ethical, reputational, or compliance issues.
3. Confirm the draft is clear and client-ready.
4. Return your verdict and notes in strict JSON:

```
{
  \"qa_notes_str\": \"newline-separated list of issues or observations\",
  \"approved_draft_str\": \"Short paragraph summarising the final validated content
ready for editing.\"
},
model: \"gpt-4.1\",
modelSettings: {
  temperature: 1,
  topP: 1,
  maxTokens: 2048,
  store: true
}
});
```

```
const editor = new Agent({
  name: \"Editor\",
  instructions: `You are the Editor preparing the final client deliverable.
```


Use the validated draft below to produce:

1. A crisp executive summary (2–3 paragraphs)
2. A slide outline (5–6 slide titles with 1-line bullets each)

Approved draft:

{{ QA & Ethics.approved_draft_str }}

Findings:

{{ Analyst.findings_str }}

Recommendation:

{{ Strategy Lead.recommendation_str }}

Plan:

{{ PMO.plan_str }}

Return strict JSON:

```
{
  \"executive_summary_str\": \"Full text of the executive summary\",
  \"slides_str\": \"newline-separated list of slide titles and brief contents\"
},
model: \"gpt-4.1\",
modelSettings: {
  temperature: 1,
  topP: 1,
  maxTokens: 2048,
  store: true
}
```

```
}  
});
```

```
const completion = new Agent({  
  name: "Completion",  
  instructions: `You are the workflow completion step.
```

Always respond with exactly this text, and nothing else:

Project signed off by Director. Deliverables approved for client release.` ,

```
  model: "gpt-4.1",  
  modelSettings: {  
    temperature: 1,  
    topP: 1,  
    maxTokens: 2048,  
    store: true  
  }  
});
```

```
const reviewNeeded = new Agent({  
  name: "Review Needed",  
  instructions: "Project NOT signed off by Director. More work is needed. Director to  
advise.",  
  model: "gpt-5",  
  modelSettings: {  
    reasoning: {  
      effort: "low"  
    },  
    store: true
```

```
}  
});
```

```
const pmDisplay = new Agent({  
  name: "PM (Display)",  
  instructions: `You are formatting the PMO report for display.  
Present the 30/60/90-day plan and key risks as readable bullet points.
```

```
### 30/60/90-Day Action Plan
```

```
- {{ PMO.plan_30d }}  
- {{ PMO.plan_60d }}  
- {{ PMO.plan_90d }}
```

```
### Key Risks
```

```
1. {{ PMO.risk_1 }}  
2. {{ PMO.risk_2 }}  
3. {{ PMO.risk_3 }}` ,
```

```
  model: "gpt-4.1",  
  modelSettings: {  
    temperature: 1,  
    topP: 1,  
    maxTokens: 2048,  
    store: true  
  }  
});
```

```
const strategyDisplay = new Agent({  
  name: "Strategy (Display)",
```

instructions: `Format the strategy summary for human review.

Strategic Options

{{ \"Strategy Lead\".options_str }}

Recommendation

{{ \"Strategy Lead\".recommendation_str }}`,

model: \"gpt-4.1\",

modelSettings: {

temperature: 1,

topP: 1,

maxTokens: 2048,

store: true

}

});

const analysisDisplay = new Agent({

name: \"Analysis (Display)\",

instructions: `You are formatting the Analyst findings for display.

Analytical Findings

{{ Analyst.findings_str }}

Suggested Charts

{{ Analyst.charts_str }}`,

model: \"gpt-4.1\",

modelSettings: {

temperature: 1,

```
topP: 1,  
maxTokens: 2048,  
store: true  
}  
});
```

```
const researchDisplay = new Agent({  
  name: "Research (Display)",  
  instructions: `You are formatting evidence for human review.`
```

Parse the JSON string below and produce a short Markdown report listing each claim with its supporting quote and URL.

Evidence (JSON string):

```
{{ Researcher.evidence_json }}`,  
model: "gpt-4.1",  
modelSettings: {  
  temperature: 1,  
  topP: 1,  
  maxTokens: 2048,  
  store: true  
}  
});
```

```
const qaDisplay = new Agent({  
  name: "QA (Display)",  
  instructions: `Format the QA report for review.`
```

QA Summary

```
{{ \"QA & Ethics\".qa_notes_str }}
```

Final Approval Statement

```
{{ \"QA & Ethics\".approved_draft_str }}`,
```

```
model: \"gpt-4.1\",
```

```
modelSettings: {
```

```
  temperature: 1,
```

```
  topP: 1,
```

```
  maxTokens: 2048,
```

```
  store: true
```

```
}
```

```
});
```

```
const reportDisplay = new Agent({
```

```
  name: \"Report (Display)\",
```

```
  instructions: `Format the final deliverables for presentation.
```

Executive Summary

```
{{ Editor.executive_summary_str }}
```

Slide Outline

```
{{ Editor.slides_str }}`,
```

```
model: \"gpt-4.1\",
```

```
modelSettings: {
```

```
  temperature: 1,
```

```
  topP: 1,
```

```
  maxTokens: 2048,
```

```
    store: true
  }
});
```

```
const scopingDisplay = new Agent({
  name: "Scoping (Display)",
  instructions: `You are formatting the scoping summary for display.
```

```
### Project Scope
```

```
**Objectives**
```

```
{{ \"Intake & Scoping\".objectives_str }}
```

```
**KPIs**
```

```
{{ \"Intake & Scoping\".kpis_str }}
```

```
**Hypotheses**
```

```
{{ \"Intake & Scoping\".hypotheses_str }}`,
```

```
  model: "gpt-4.1",
```

```
  modelSettings: {
```

```
    temperature: 1,
```

```
    topP: 1,
```

```
    maxTokens: 2048,
```

```
    store: true
```

```
  }
```

```
});
```

```
const approvalRequest = (message: string) => {
```

```
// TODO: Implement  
return true;  
}
```

```
const approvalRequest1 = (message: string) => {
```

```
// TODO: Implement  
return true;  
}
```

```
const approvalRequest2 = (message: string) => {
```

```
// TODO: Implement  
return true;  
}
```

```
type WorkflowInput = { input_as_text: string };
```

```
// Main code entrypoint
```

```
export const runWorkflow = async (workflow: WorkflowInput) => {  
  return await withTrace("Enterprise Vibe Consulting Demo", async () => {  
    const state = {  
  
    };  
    const conversationHistory: AgentInputItem[] = [  
      {  
        role: "user",
```



```

    content: [
      {
        type: "input_text",
        text: workflow.input_as_text
      }
    ]
  }
];

const runner = new Runner({
  traceMetadata: {
    __trace_source__: "agent-builder",
    workflow_id: "wf_68f4e8d79dc881908baaffaeb44e1c4e0cdde14d643a3b23"
  }
});

const intakeScopingResultTemp = await runner.run(
  intakeScoping,
  [
    ...conversationHistory
  ]
);

conversationHistory.push(...intakeScopingResultTemp.newItems.map((item) =>
item.rawItem));

if (!intakeScopingResultTemp.finalOutput) {
  throw new Error("Agent result is undefined");
}

const intakeScopingResult = {

```

```

    output_text: JSON.stringify(intakeScopingResultTemp.finalOutput),
    output_parsed: intakeScopingResultTemp.finalOutput
  };
  const scopingDisplayResultTemp = await runner.run(
    scopingDisplay,
    [
      ...conversationHistory
    ]
  );
  conversationHistory.push(...scopingDisplayResultTemp.newItems.map((item) =>
    item.rawItem));

```

```

if (!scopingDisplayResultTemp.finalOutput) {
  throw new Error("Agent result is undefined");
}

```

```

const scopingDisplayResult = {
  output_text: scopingDisplayResultTemp.finalOutput ?? ""
};
const approvalMessage = `### Project Scope Review

```

****Objectives****

```

{{ \"Intake & Scoping\".objectives_str }}

```

****KPIs****

```

{{ \"Intake & Scoping\".kpis_str }}

```

****Hypotheses****

```
{{ \"Intake & Scoping\".hypotheses_str }}
```

Please review the proposed scope.

Select ****Approve**** to proceed to the Research phase or ****Reject**** to request clarification.`;

```
if (approvalRequest(approvalMessage)) {  
  const researcherResultTemp = await runner.run(  
    researcher,  
    [  
      ...conversationHistory  
    ]  
  );  
  conversationHistory.push(...researcherResultTemp.newItems.map((item) =>  
item.rawItem));  
  
  if (!researcherResultTemp.finalOutput) {  
    throw new Error("Agent result is undefined");  
  }  
  
  const researcherResult = {  
    output_text: JSON.stringify(researcherResultTemp.finalOutput),  
    output_parsed: researcherResultTemp.finalOutput  
  };  
  const researchDisplayResultTemp = await runner.run(  
    researchDisplay,  
    [  
      ...conversationHistory  
    ]  
  );
```

```

);

conversationHistory.push(...researchDisplayResultTemp.newItems.map((item) =>
item.rawItem));

if (!researchDisplayResultTemp.finalOutput) {
  throw new Error("Agent result is undefined");
}

const researchDisplayResult = {
  output_text: researchDisplayResultTemp.finalOutput ?? ""
};

const analystCsvDataInsightsSimulatedResultTemp = await runner.run(
  analystCsvDataInsightsSimulated,
  [
    ...conversationHistory
  ]
);

conversationHistory.push(...analystCsvDataInsightsSimulatedResultTemp.newItems.m
ap((item) => item.rawItem));

if (!analystCsvDataInsightsSimulatedResultTemp.finalOutput) {
  throw new Error("Agent result is undefined");
}

const analystCsvDataInsightsSimulatedResult = {
  output_text: analystCsvDataInsightsSimulatedResultTemp.finalOutput ?? ""
};

const analysisDisplayResultTemp = await runner.run(

```

```
analysisDisplay,  
[  
  ...conversationHistory  
]  
);  
conversationHistory.push(...analysisDisplayResultTemp.newItems.map((item) =>  
item.rawItem));
```

```
if (!analysisDisplayResultTemp.finalOutput) {  
  throw new Error("Agent result is undefined");  
}
```

```
const analysisDisplayResult = {  
  output_text: analysisDisplayResultTemp.finalOutput ?? ""  
};  
const strategyLeadResultTemp = await runner.run(  
  strategyLead,  
  [  
    ...conversationHistory  
  ]  
);  
conversationHistory.push(...strategyLeadResultTemp.newItems.map((item) =>  
item.rawItem));
```

```
if (!strategyLeadResultTemp.finalOutput) {  
  throw new Error("Agent result is undefined");  
}
```

```
const strategyLeadResult = {
```

```
    output_text: strategyLeadResultTemp.finalOutput ?? ""
  };
  const strategyDisplayResultTemp = await runner.run(
    strategyDisplay,
    [
      ...conversationHistory
    ]
  );
  conversationHistory.push(...strategyDisplayResultTemp.newItems.map((item) =>
    item.rawItem));
```

```
if (!strategyDisplayResultTemp.finalOutput) {
  throw new Error("Agent result is undefined");
}
```

```
const strategyDisplayResult = {
  output_text: strategyDisplayResultTemp.finalOutput ?? ""
};
const pmoResultTemp = await runner.run(
  pmo,
  [
    ...conversationHistory
  ]
);
conversationHistory.push(...pmoResultTemp.newItems.map((item) =>
  item.rawItem));
```

```
if (!pmoResultTemp.finalOutput) {
  throw new Error("Agent result is undefined");
}
```

```
}
```

```
const pmoResult = {
```

```
  output_text: pmoResultTemp.finalOutput ?? ""
```

```
};
```

```
const pmDisplayResultTemp = await runner.run(
```

```
  pmDisplay,
```

```
  [
```

```
    ...conversationHistory
```

```
  ]
```

```
);
```

```
  conversationHistory.push(...pmDisplayResultTemp.newItems.map((item) =>
item.rawItem));
```

```
if (!pmDisplayResultTemp.finalOutput) {
```

```
  throw new Error("Agent result is undefined");
```

```
}
```

```
const pmDisplayResult = {
```

```
  output_text: pmDisplayResultTemp.finalOutput ?? ""
```

```
};
```

```
const qaEthicsResultTemp = await runner.run(
```

```
  qaEthics,
```

```
  [
```

```
    ...conversationHistory
```

```
  ]
```

```
);
```

```
  conversationHistory.push(...qaEthicsResultTemp.newItems.map((item) =>
item.rawItem));
```

```

if (!qaEthicsResultTemp.finalOutput) {
  throw new Error("Agent result is undefined");
}

const qaEthicsResult = {
  output_text: qaEthicsResultTemp.finalOutput ?? ""
};

const qaDisplayResultTemp = await runner.run(
  qaDisplay,
  [
    ...conversationHistory
  ]
);

conversationHistory.push(...qaDisplayResultTemp.newItems.map((item) =>
item.rawItem));

if (!qaDisplayResultTemp.finalOutput) {
  throw new Error("Agent result is undefined");
}

const qaDisplayResult = {
  output_text: qaDisplayResultTemp.finalOutput ?? ""
};

const approvalMessage1 = `### QA Review — Approval Required

**QA Notes**

{{ \"QA & Ethics\".qa_notes_str }}

```


****Validated Draft (Summary)****

{{ \"QA & Ethics\".approved_draft_str }}

Please review the QA findings and validated draft before proceeding to editing.

- ****Approve**** → Send to Editor for final polish.
- ****Reject**** → Return to QA & Ethics for revision.`;

```
if (approvalRequest2(approvalMessage1)) {  
  const editorResultTemp = await runner.run(  
    editor,  
    [  
      ...conversationHistory  
    ]  
  );  
  conversationHistory.push(...editorResultTemp.newItems.map((item) =>  
item.rawItem));  
  
  if (!editorResultTemp.finalOutput) {  
    throw new Error("Agent result is undefined");  
  }  
  
  const editorResult = {  
    output_text: editorResultTemp.finalOutput ?? ""  
  };  
  const reportDisplayResultTemp = await runner.run(  
    reportDisplay,
```

```
[
    ...conversationHistory
]
);

conversationHistory.push(...reportDisplayResultTemp.newItems.map((item) =>
item.rawItem));
```

```
if (!reportDisplayResultTemp.finalOutput) {
    throw new Error("Agent result is undefined");
}
```

```
const reportDisplayResult = {
    output_text: reportDisplayResultTemp.finalOutput ?? ""
};
```

```
const approvalMessage2 = ` Please review the final deliverables before sign-off.
```

Executive Summary:

```
{{ Editor.executive_summary_str }}
```

Slide Outline:

```
{{ Editor.slides_str }}
```

Confirm that the report meets the firm's quality standards and is ready for client delivery.

Click ****Approve**** to complete the engagement, or ****Reject**** to send back to QA & Ethics for revision.`;

```
if (approvalRequest1(approvalMessage2)) {
```

```

const completionResultTemp = await runner.run(
  completion,
  [
    ...conversationHistory
  ]
);

conversationHistory.push(...completionResultTemp.newItems.map((item) =>
item.rawItem));

if (!completionResultTemp.finalOutput) {
  throw new Error("Agent result is undefined");
}

const completionResult = {
  output_text: completionResultTemp.finalOutput ?? ""
};

return completionResult;
} else {
  const reviewNeededResultTemp = await runner.run(
    reviewNeeded,
    [
      ...conversationHistory
    ]
  );

  conversationHistory.push(...reviewNeededResultTemp.newItems.map((item) =>
item.rawItem));

  if (!reviewNeededResultTemp.finalOutput) {
    throw new Error("Agent result is undefined");
  }
}

```

```
}

const reviewNeededResult = {
  output_text: reviewNeededResultTemp.finalOutput ?? ""
};

return reviewNeededResult;
}
} else {
  return qaDisplayResult;
}
} else {
  return scopingDisplayResult;
}
});
}
```